

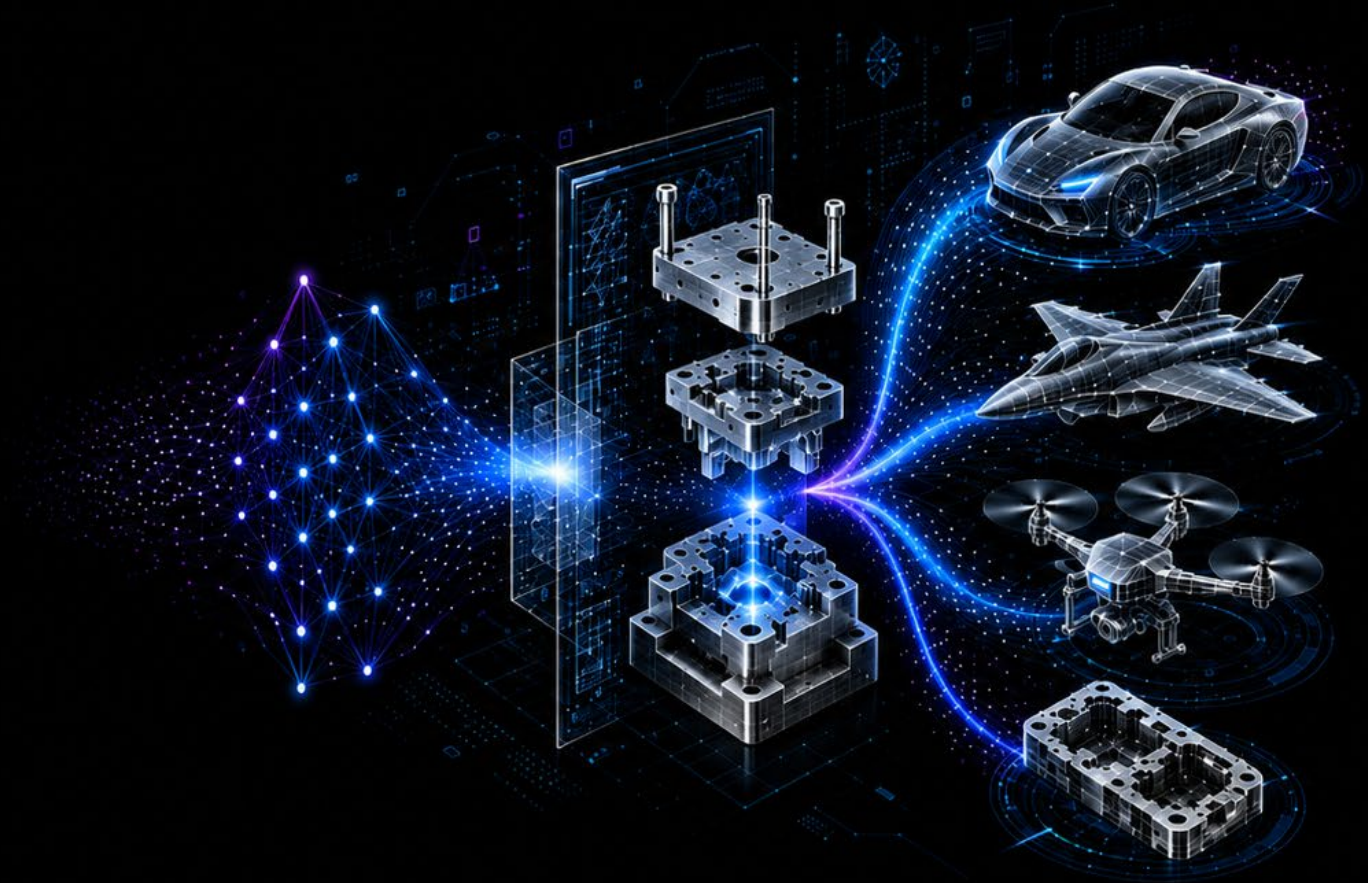
AI STUDY 8장

성능최적화

안동균

(지도교수: 양선웅)

한양대학교 ERICA 로봇공학과



Intelligent Design
for Engineering
Applications Lab

정규화, 드롭아웃, 조기 종료

- 하이퍼파라미터를 이용한 성능 최적화의 방법으로 배치 정규화, 드롭아웃, 조기종료가 있음

IMDB(7장) 데이터로 학습한 결과

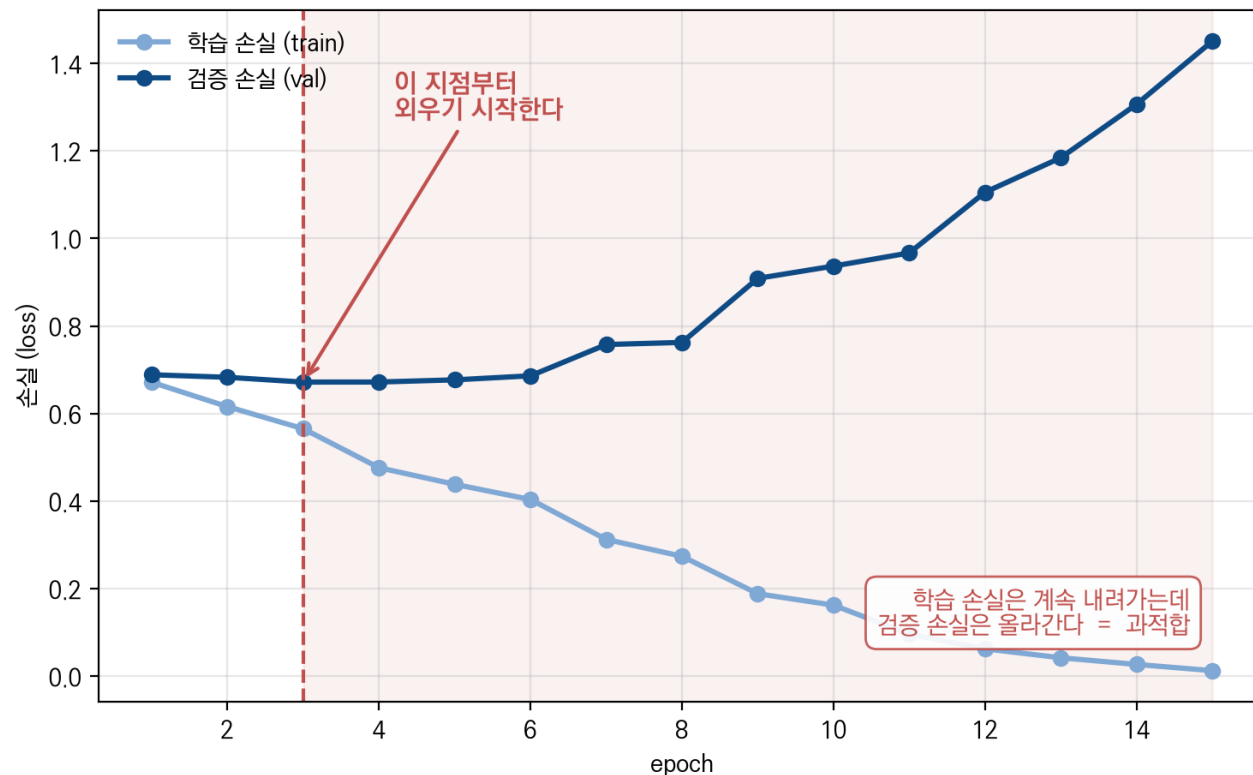
IMDB 실험데이터(stanfordnlp/imdb) 실제 로드 → 학습 2000개 / 검증 2000개
vocab 5000, maxlen 200, Embedding(64) → LSTM(128) → Linear(2)
Adam lr=1e-3, batch 64, dropout 0, LayerNorm 없음, 조기 종료 없음, GPU 15에폭

❖ 저번 시간에 mnist와 imdb 데이터를 이용한 RNN코드를 돌렸을때

❖ 또한 데이터가 양 대비 파라미터수(모델 용량)가 많거나 학습을 과하게 오래 돌리면 과적합이 일어남-데이터를 일반화하는 대신 외우게 된다

❖ 과적합?

드롭아웃이 없으면 — 모델이 데이터를 외운다



정규화를 이용한 성능 최적화

- 배치 정규화(Batch normalization)
- 레이어 정규화(Layer normalization)

배치 정규화

- 데이터가 한쪽에 쏠려있으면 제대로 학습하기가 어려움(vanishing gradient 해결과 비선형성을 살리는) → 적절히 재배치를 해보자
- 배치 정규화: 각 층의 출력을 미니배치 단위로 평균 0, 분산 1로 맞춰 주는 기법

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

μ_B, σ_B^2 미니배치안에서의 평균과 분산

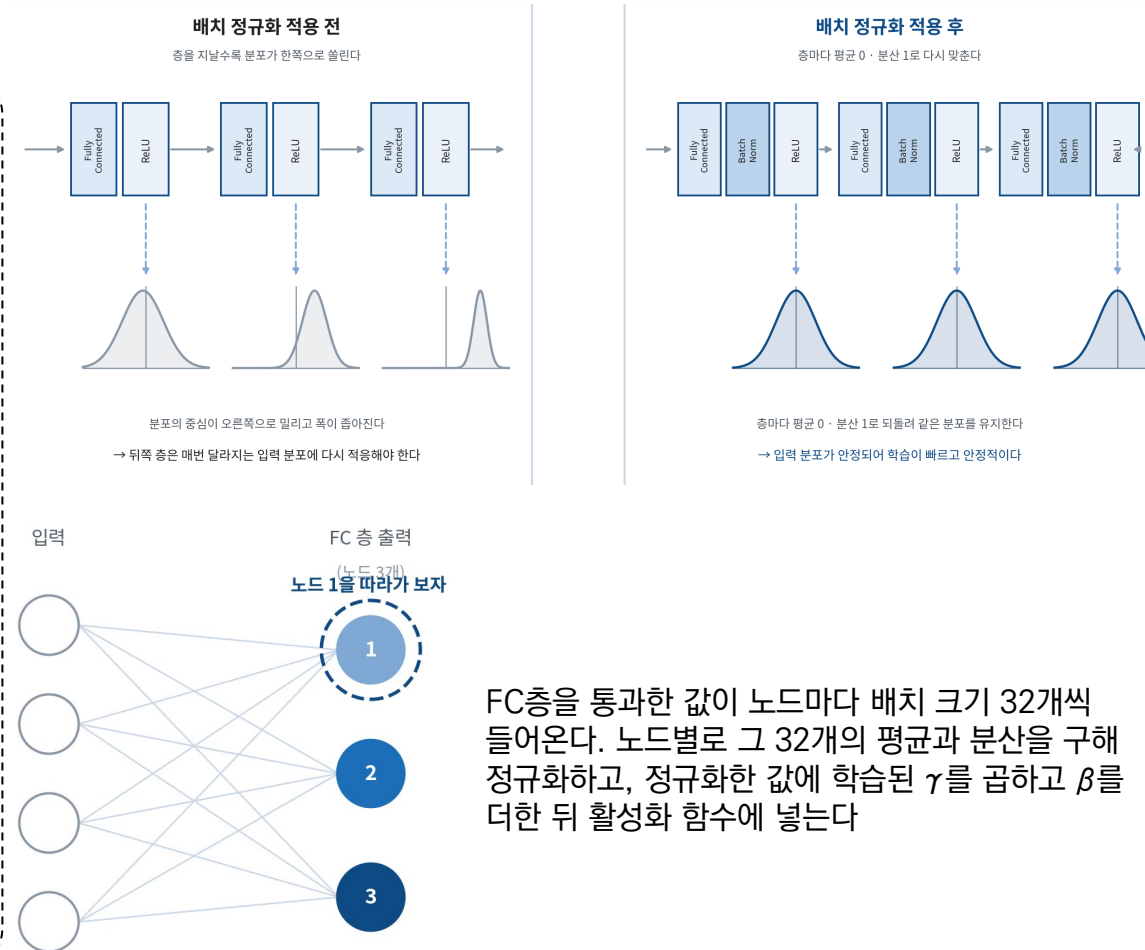
γ, β 는 학습해야 할 파라미터 $\frac{\partial L}{\partial \gamma}, \frac{\partial L}{\partial \beta}$ 학습하며 업데이트함

뭉쳐있거나 넓게 분포된 데이터를 평균 0 분산 1로 정규화 시키고 그 값을 어디로 재배치 시킬지를 학습시키자.
정규화만 하고 재배치를 하지 않으면 비선형성을 잃게됨

- Train과 test를 다르게 학습 때는 현재 배치의 통계를 쓰고, 평가 때는 학습 중 누적한 이동평균을 써야 함.
- 배치사이즈에 따라 영향을 받는다는 단점이 있다
배치사이즈가 매우 작으면 부정확해짐

※정규화(normalization): 데이터 범위를 원하는 범위로 제한하는것

$$\hat{x} = \frac{x - \mu}{\sigma}$$



Part 1 Normalization

Part 2 Drop out

Part 3 Early stopping

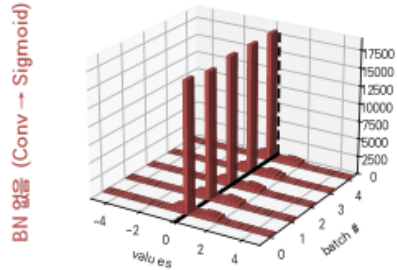
Part 4 Conclusion

배치 정규화 적용 전 후 비교 실험

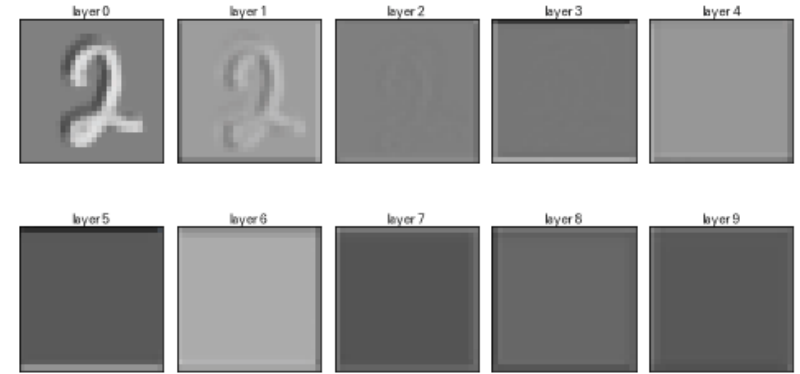
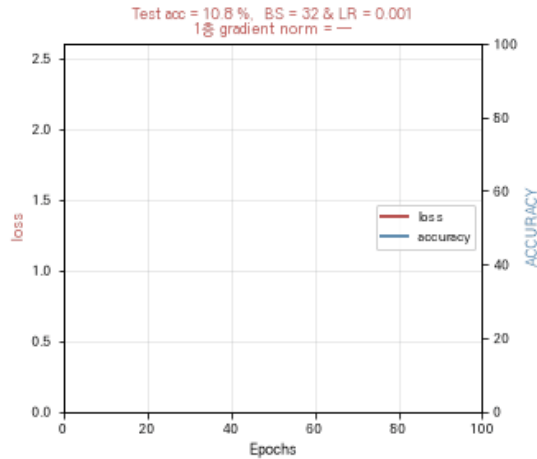
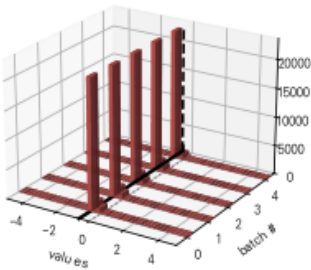
- Mnist 데이터를 이용하여 BN적용 전 후 차이를 각 레이어마다 비교

Sigmoid 사용 — Epoch: 0, # of conv block = 10, # of channels = 1

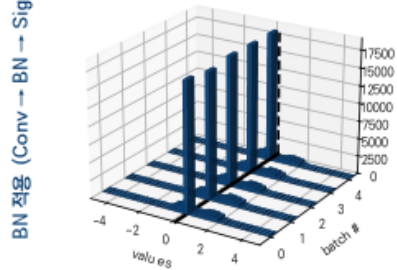
first layer 평균 +0.15 · 표준편차 0.52



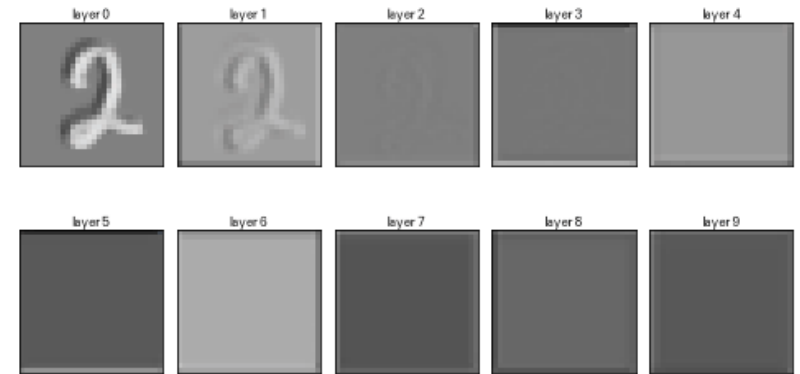
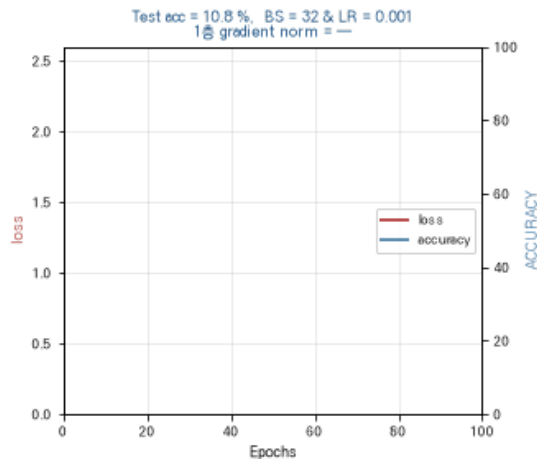
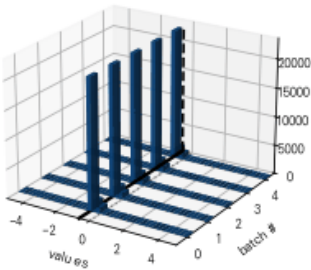
last layer 평균 -0.60 · 표준편차 0.11



first layer 평균 +0.15 · 표준편차 0.52



last layer 평균 -0.60 · 표준편차 0.11



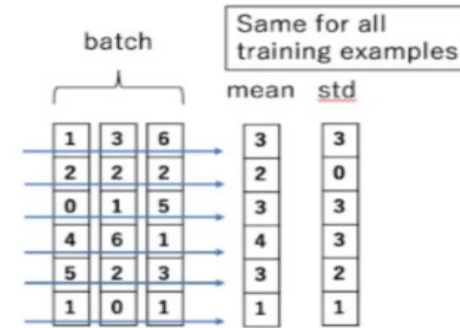
배치 정규화는 각 층의 출력을 평균 0·분산 1로 되돌려 활성화함수의 포화 구간을 벗어나게 한다(기울기 소실 완화). 다만 이렇게만 하면 값이 0 근처에 몰려 비선형성을 잃으므로, 학습 가능한 $\gamma \cdot \beta$ 로 얼마나 늘리고 어디로 옮길지를 모델이 알아서 찾게 한다

레이어 정규화(layer normalization)

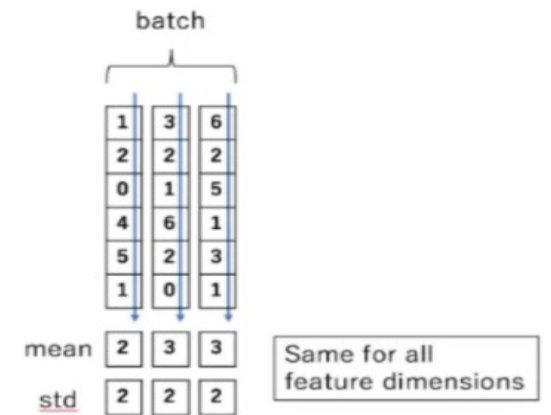
- 7장 IMDB 데이터를 이용한 코드를 통해 배치 정규화 실험을 해보려 했지만 결과가 잘 나오지 않았다(패딩때문)
- 그럴때는 보통 layer normalization을 적용한다고 하여 찾아봄

- ❖ 레이어 정규화: 한 레이어에 배치된 모든 노드에 들어가는 값 각각에 대해 평균(mean)과 표준편차(std)를 계산해서 정규화를 실행
- ❖ 배치 정규화와의 차이는 무엇으로 평균 분산을 구할거냐만 다르고 나머지는 동일
- ❖ 배치 사이즈에 영향을 받지않으므로 시계열데이터에 잘 어울림
→RNN 계열
- ❖ 7장 LSTM 코드에 배치 정규화를 적용했음에도 결과값이 잘 안나왔던 이유는 길이가 짧은 데이터는 패딩을 해주므로 batch 끼리 정규화 해주는 의미가 사라진다

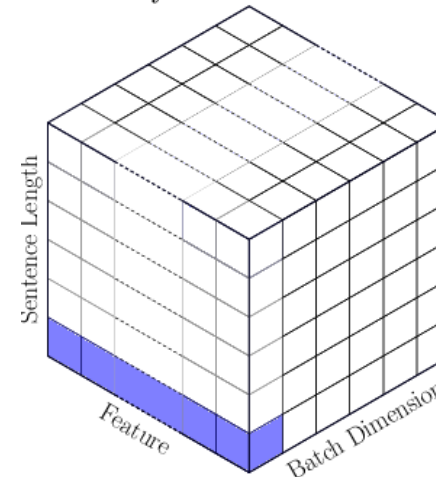
Batch Normalization



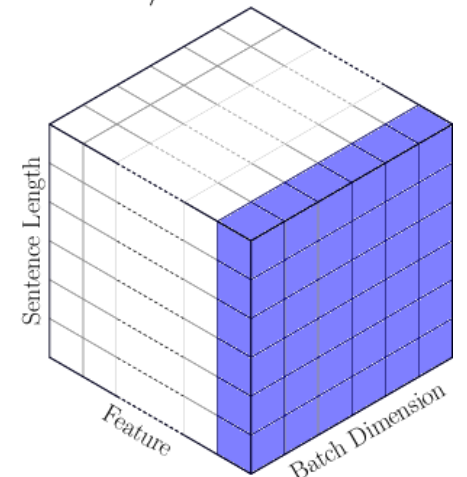
Layer Normalization



Layer Normalization



Batch/Power Normalization

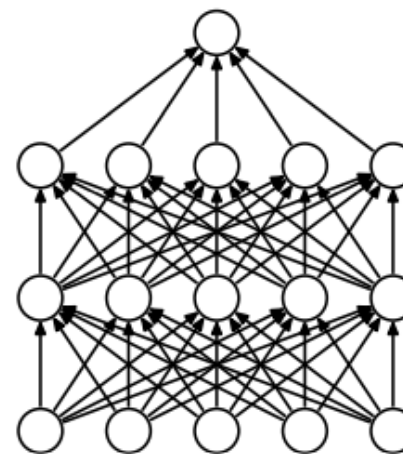


드롭아웃을 이용한 성능 최적화

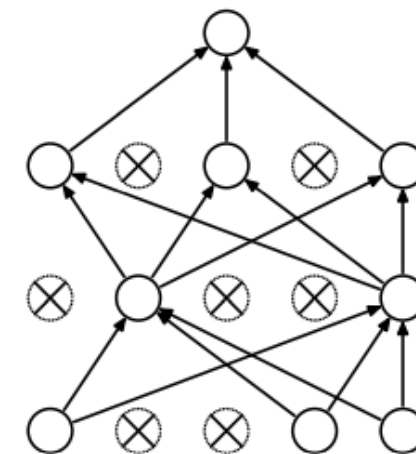
- 왜 드롭아웃을 쓰는가? – 과적합을 방지하기 위하여

드롭아웃

- ❖ 과적합이란(overfitting) – 모델이 훈련 데이터를 학습할때 일반화 하지 않고 데이터 자체를 외워버리는 현상
→ 모델 수에 비해 데이터가 작을때 일어남
→ 이러한 과적합을 방지하기 위해 만든 기법이 Drop out
- ❖ 드롭아웃은 매 학습 스텝마다 설정한 확률 p 로 뉴런을 끄는 기법
훈련할때 일정 비율의 뉴런만 사용하고 나머지 뉴런에 해당하는 가중치는 업데이트하지 않는 방법 – 꺼진 노드는 신호를 전달하지 않으므로 지나친 학습을 방지한다(p 는 hyperparameter)
- ❖ 일부 노드가 없더라도 loss를 잘 줄일수 있게 학습을 시키자
- ❖ 학습때는 무작위로 노드를 끄며 학습하다 평가시엔 모든 노드를 사용하여 출력하되 드롭아웃비율 p 를 곱해줘서 보정해준다
드롭아웃은 학습 때만 동작해야 한다 – PyTorch에서 `model.train()` / `model.eval()` 로 모드를 전환해야 하며, 평가 전에 `eval()`을 빠뜨리면 노드가 계속 꺼진 채 측정되어 성능이 실제보다 낮게 나온다.
- ❖ 각 뉴런이 더 독립적이고 의미있는 특징을 뽑게 학습



(a) Standard Neural Net



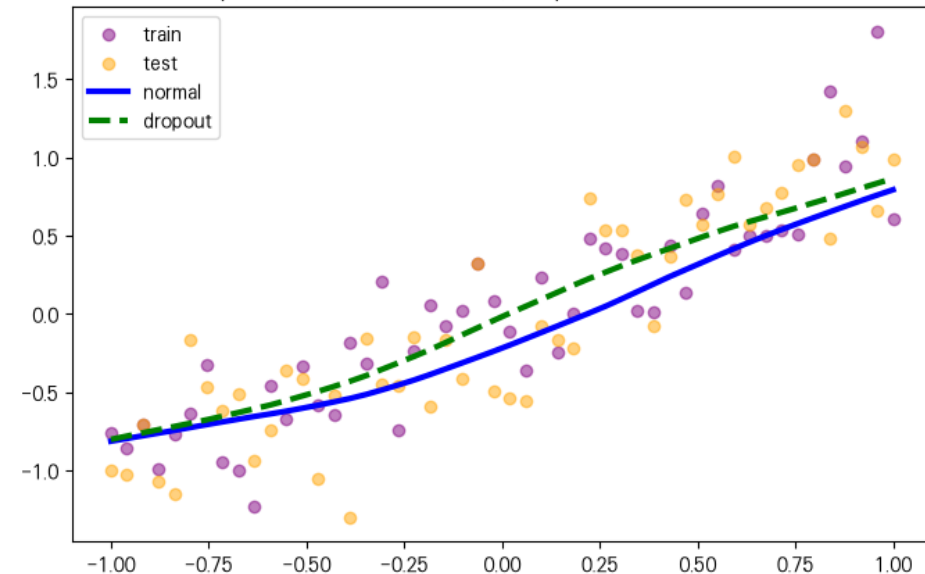
(b) After applying dropout.

- 학습 시 — 매 단계 무작위로 다른 노드를 끄
- 평가 시 — 모든 노드 사용, 드롭아웃 비율로 보정
- 효과 — 특정 노드 의존을 막아 과적합 억제

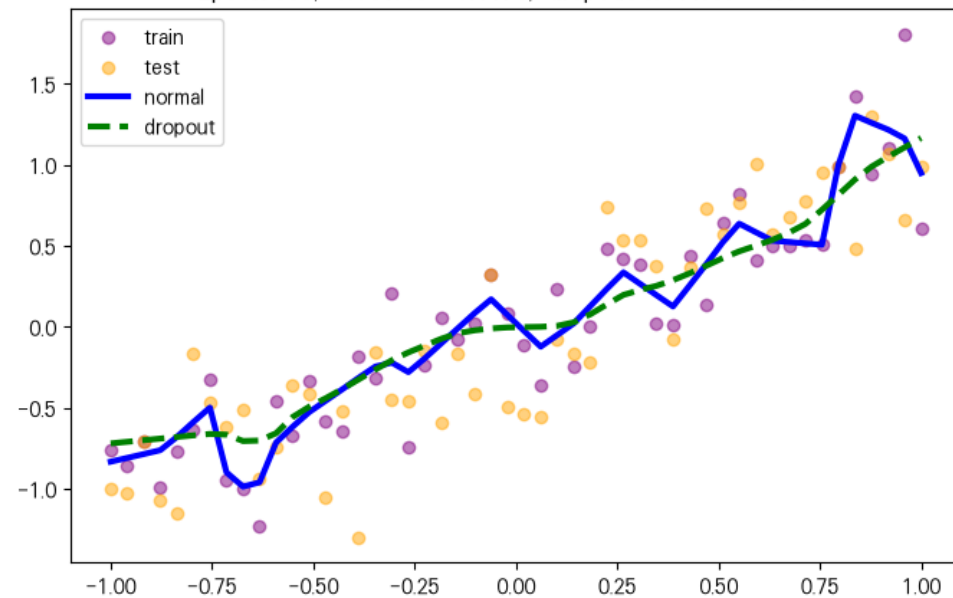
교과서 속 예제를 통한 드롭아웃 학습

- 파란 실선 normal = 기법 없음 (정규화 아님)
 - 초록 점선 dropout = 같은 모델 + 드롭아웃만
 - 보라 점 = 학습 데이터 / 노랑 점 = 테스트 데이터
-
- 결과 그래프에서 훈련 횟수가 늘어날수록 파란색실선(드롭아웃 적용이 안된 모델)은 가장자리의 자주색 점들을 찾아가고 있다.
→ 자주색 선은 훈련 데이터셋을 의미함 → 이는 과적합 현상을 보이고 있다
 - 과적합이 발생하는 모델은 훈련 데이터에 대한 정확도는 높지만 검증 데이터나 테스트 데이터와 같은 새로운 데이터에 대해서는 정확도가 낮은 문제를 보인다
 - 이와 같은 과적합 문제를 방지하기 위하여 드롭아웃을 사용한 초록색 점선은 과적합 현상이 일어나지않고 낮은 오차를 보여주고있다.

Epoch 0, test loss = 0.1118, dropout test loss = 0.1038



Epoch 800, test loss = 0.1176, dropout test loss = 0.1168



7장 코드에 드롭아웃을 적용

- 7장 RNN/LSTM을 이해하기 위해 IMDB 리뷰 데이터 학습 코드에 드롭아웃을 적용

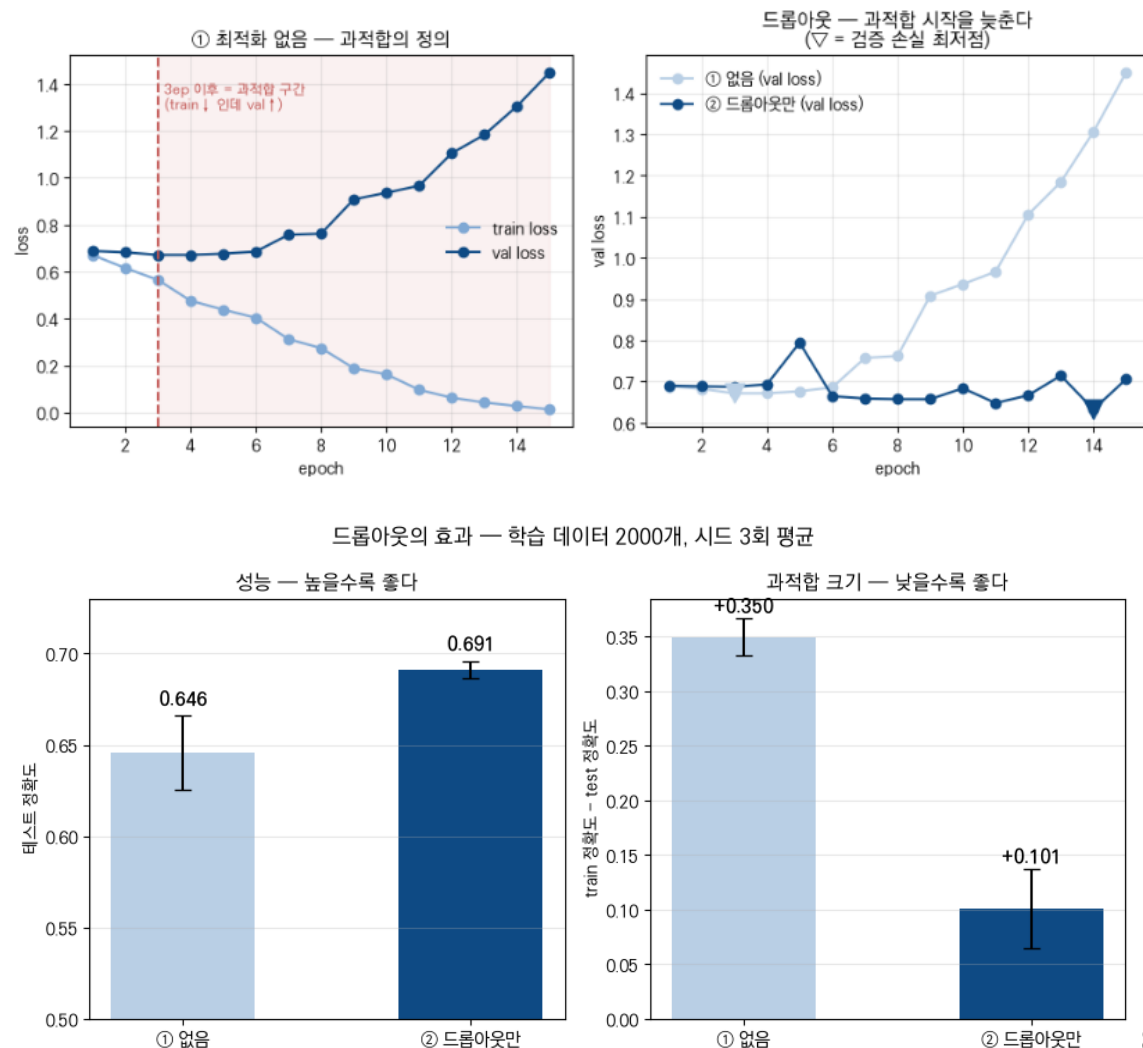
Part 1
Normalization

Part 2
Drop out

Part 3
Early
stopping

Part 4
Conclusion

- ❖ 실험을 위해 약 모델용량에 비해 작은 데이터(2000개)를 준비
→ 과적합 유도
- ❖ 데이터 · 전처리 — IMDB 영화 리뷰(긍정/부정 2진 분류)
- ❖ 학습 2,000 / 검증 2,000 / 테스트 25,000개
- ❖ 어휘 사전: 학습 텍스트에서 자주 쓰인 상위 5,000단어만 사용 (나머지는 <unk> 처리)
- ❖ 길이 200토큰으로 고정 — 리뷰를 앞 200단어까지만 자르고, 짧으면 뒤를 0으로 채움 (길이가 제각각이면 배치로 묶을 수 없기 때문)
- ❖ 학습 Adam lr 0.001, 배치 64, 15에폭, 시드 3회 평균
- ❖ 드롭아웃 0.5를 임베딩 출력과 분류기 직전 두 곳에 적용
두곳에 적용한 이유는 최종 특징 벡터에만 드롭아웃을 걸면 쓸수있는 노드수가 작아지므로 효과를 극대화 시키기 위해 두 곳 다 적용

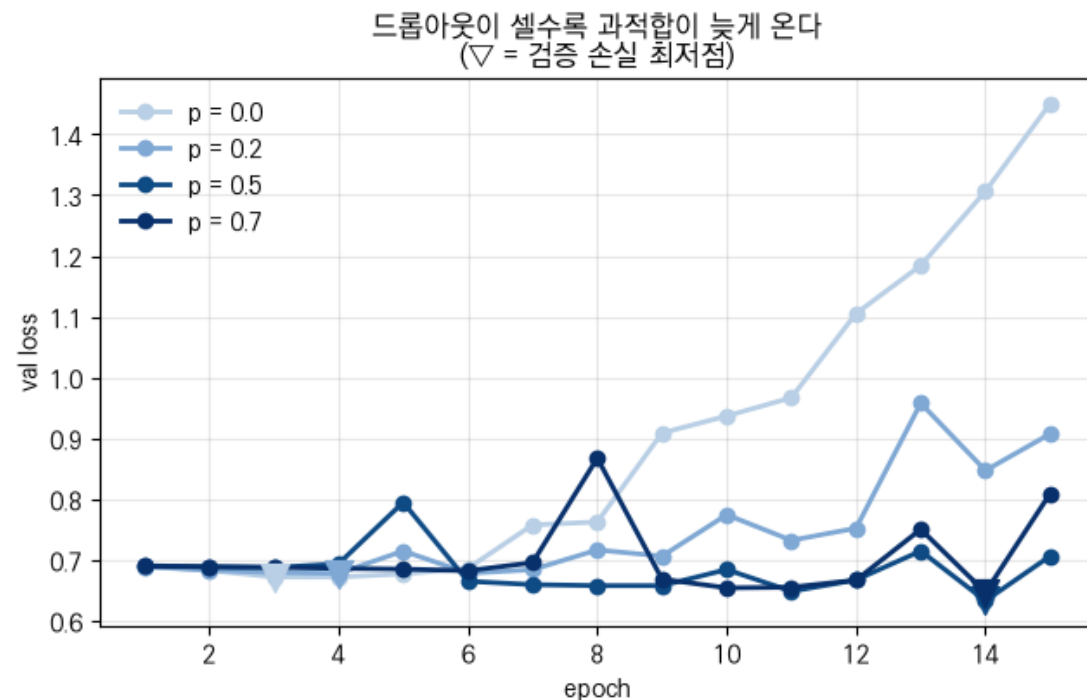
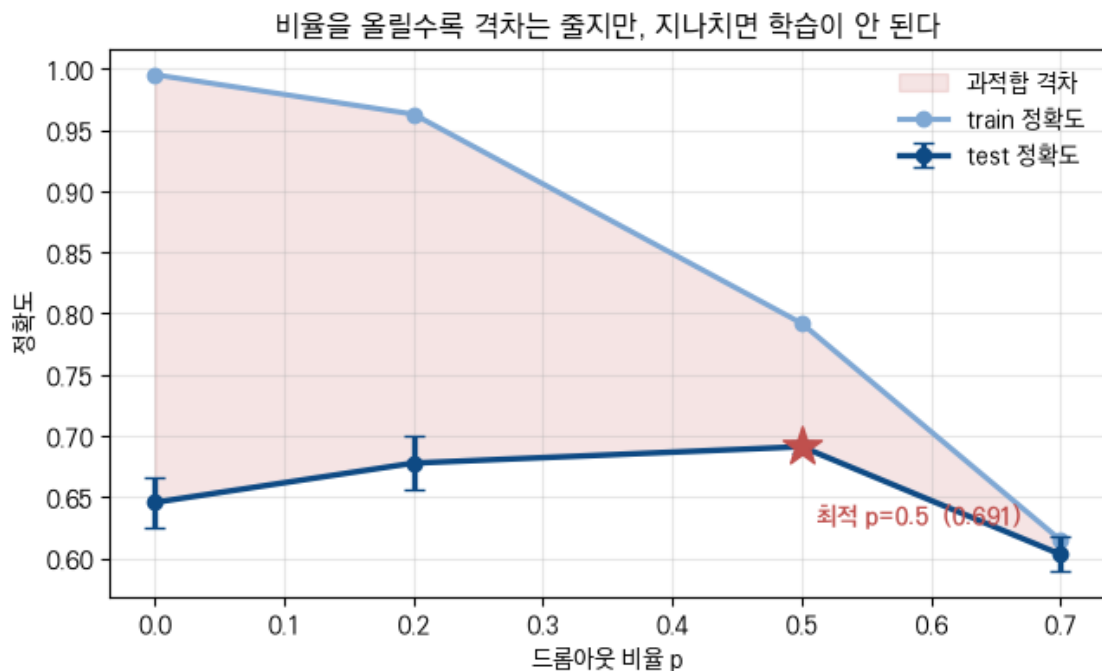


7장 코드에 드롭아웃을 적용

- 7장 RNN/LSTM을 이해하기 위해 IMDB 리뷰 데이터 학습 코드에 드롭아웃을 적용

- 드롭아웃 비율별로 얼마나 바꿀까?
- 드롭아웃 비율 `torch.nn.Dropout(0.X)` 을 무작정 늘린다고 해서 정확도가 계속 올라가진 않는것을 볼수있다.

드롭아웃 비율 스윙 — 학습 2000개, 시드 3회 평균



Part 1
Normalization

Part 2
Drop out

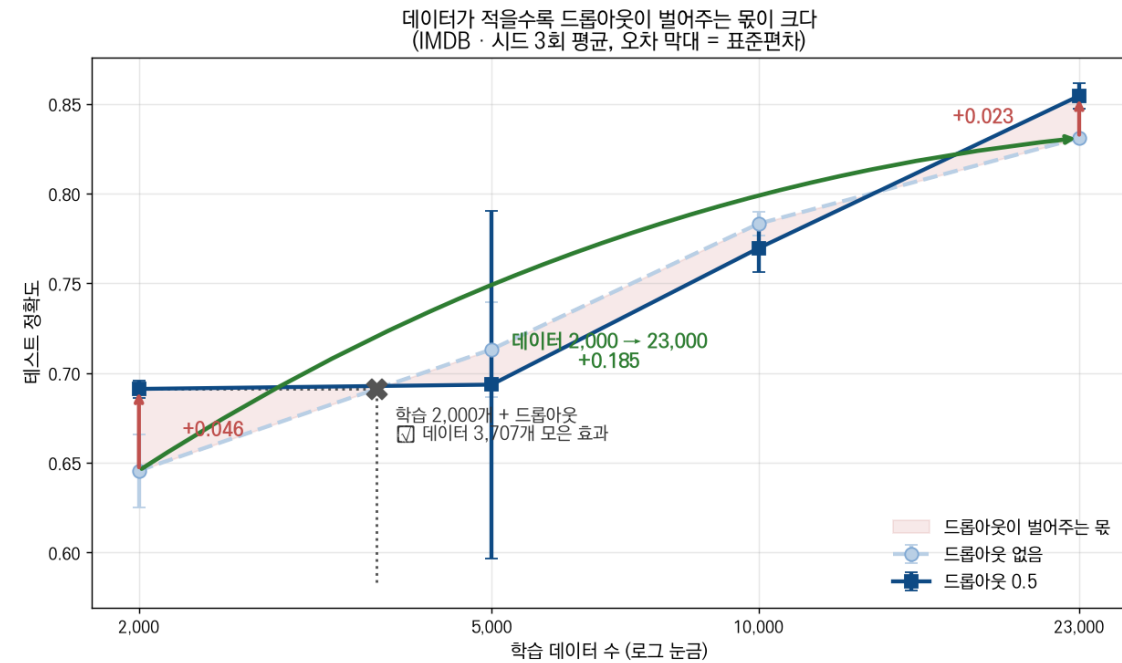
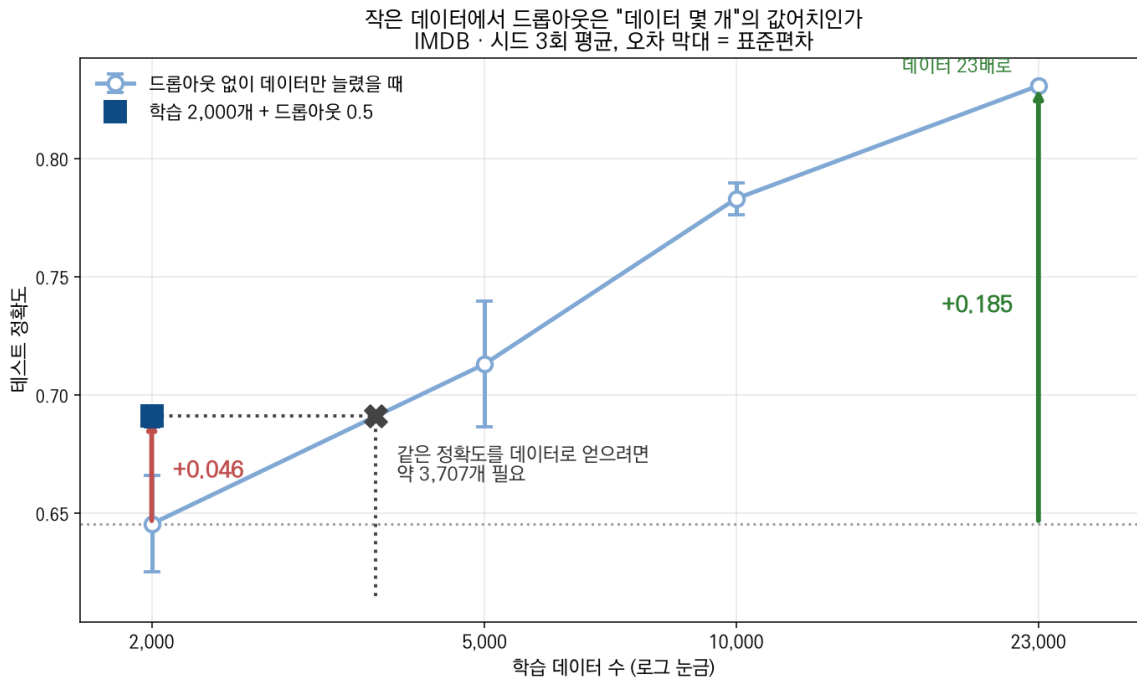
Part 3
Early
stopping

Part 4
Conclusion

■ 모델 용량 대비 학습데이터가 충분하다면?

- 과적합을 유도하기 위해 모델대비 적은 데이터를 넣지 않고 원래 있던 데이터 25000개를 다 넣었을때 결과 비교

학습데이터를 25000개로 늘렸을때 정확도가 훨씬 높지만 작은 데이터 밖에 없는 상황에서 정확도를 올리는 기법으로 드롭아웃은 좋은 선택이다



조기 종료를 이용한 과적합 회피

- 조기종료는 검증 데이터셋에 대한 오차가 증가하는 시점에서 학습을 멈추도록 조정한다
- 정해진 에폭보다 일찍 학습을 종료하는것

조기종료

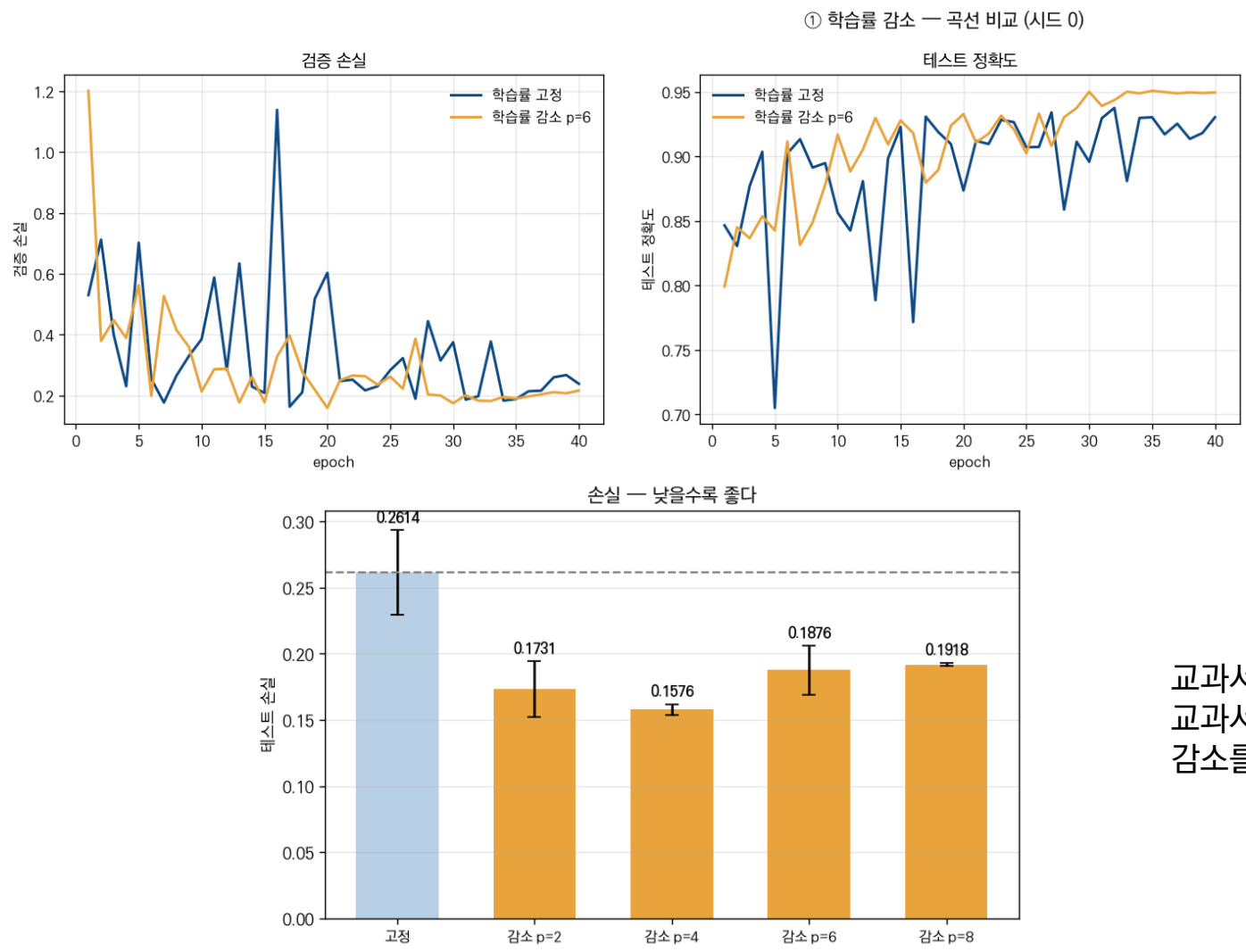
- ❖ 실제 일반화 성능을 확인하기 위해 훈련 데이터와 별도로 검증 데이터를 준비하고 매 에폭마다 검증 데이터에 대한 오차를 측정
- ❖ 과적합이 발생하기전까지 학습에 대한 오차와 검증에 대한 오차 모두 감소하지만, 과적합 발생시 훈련데이터셋 오차는 계속 줄어드는 반면 검증 데이터셋에 대한 오차는 증가함
- ❖ 조기종료는 검증 데이터셋에 대한 오차가 증가하는 시점에서 학습을 멈추도록 조정한다
- ❖ 교과서 속 예제를 통해서 조기종료를 적용했을때 정확도 차이를 이해할수있다.
- ❖ 입력 데이터는 hotdog vs not-hotdog 분류

교과서속 예제코드

- ❖ 모델은 뉴럴네트워크 직접 구축이 아닌 사전학습된 ResNet50을 사용
- ❖ Patience: 검증 오차가 한번 올랐다고 바로 조기종료 하면 안되기 때문에 얼마나 기다려줄건지를 정하는 값
→함께patience=N 일때 연속해서 N번 개선이 없으면 그때 종료
- ❖ 파이토치에서 조기종료와 자주 사용되는것이 학습률감소 학습률을 고정하지 않고 학습이 진행되는 과정에서 조금씩 낮추어 주는 튜닝기법이다
- ❖ 학습률 스케줄러(learning rate scheduler)를 이용하는데, 주어진 patience 횟수만큼 검증 데이터셋 오차 감소가 없으면 주어진 factor만큼 학습률을 감소시켜 모델 학습의 최적화를 돕는다
factor: 학습률 낮출때 곱하는 비율 $new_lr = lr \times factor$
- ❖ 학습률을 낮추는 이유? → 개선이 멈추고 patience만큼 검증 오차감소가 정체되면 지금 학습률로는 세밀한 조정이 안되는 뜻이므로 학습률을 낮춰(보폭을 줄이는 느낌) 학습을 하고 그래도 오차감소에 개선이 없으면 조기종료를 한다
- ❖ 보통 스케줄러 patience를 조기종료 patience보다 낮게 설정

■ 학습률 감소가 어떤 효과를 보이는지에 대한 결과

- 교과서 예제코드를 거의 비슷하게 구현하고 데이터를 cat vs dog 5000장을 입력함, 아래 그래프는 학습률 감소에 대한 결과

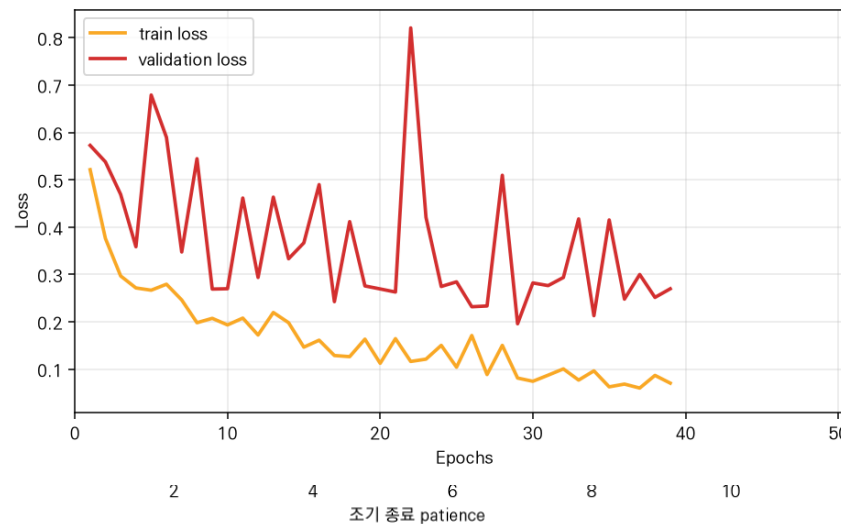
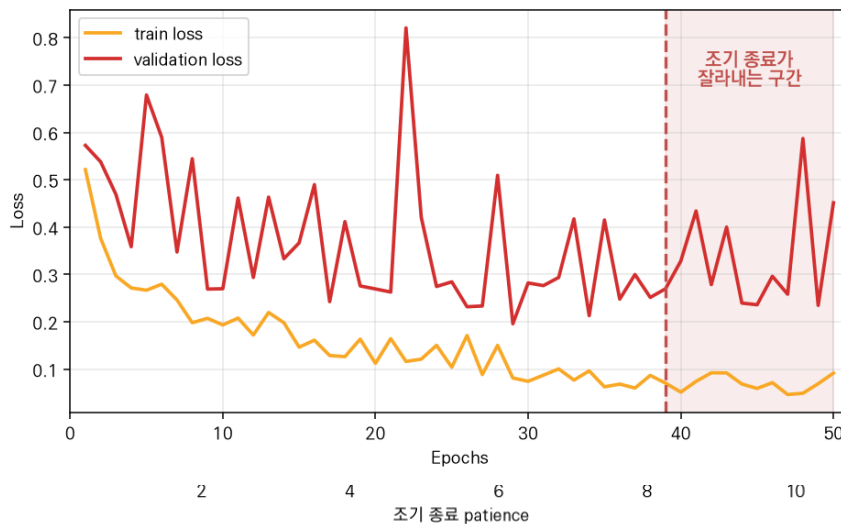
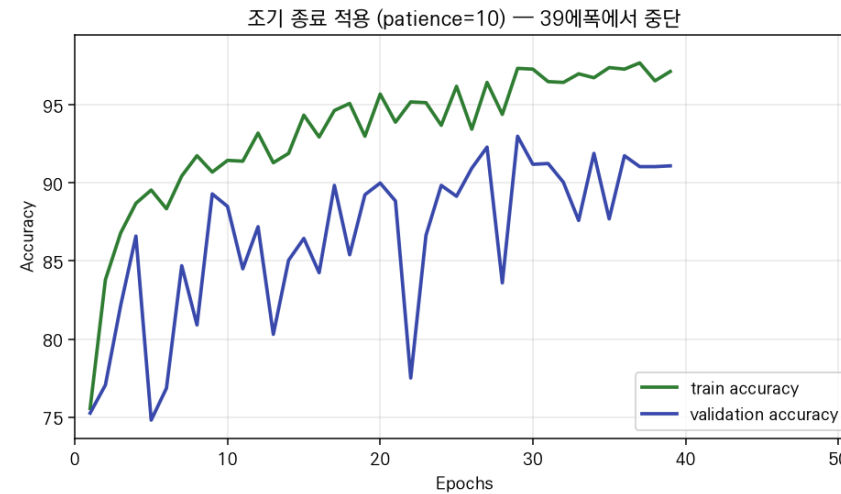
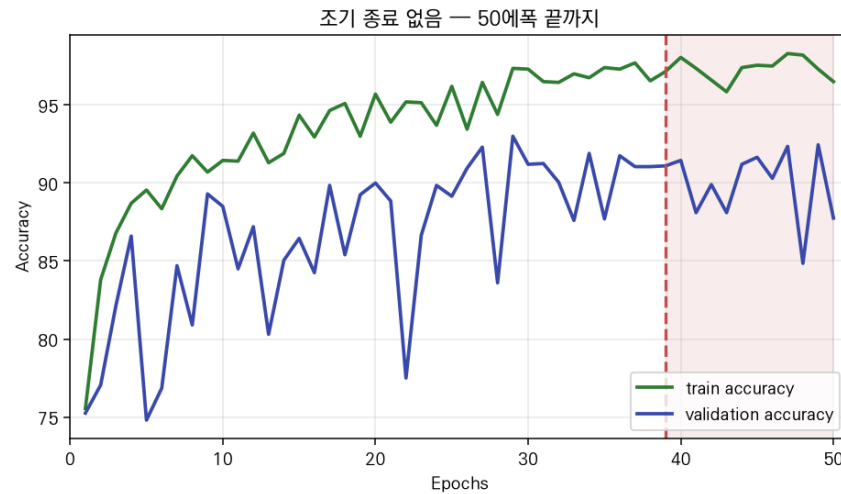


교과서와 같은 코드에 데이터만 다르게 넣어서 그런지 교과서와 완벽히 똑같이 나오진 않았지만 학습률 감소를 적용했을때 효과를 보이는 모습을 볼수있다

조기종료가 어떤 효과를 보이는지에 대한 결과

- 교과서 예제코드를 거의 비슷하게 구현하고 데이터를 cat vs dog 5000장을 입력함

조기 종료 적용 전 / 후 — 개·고양이 분류 (ResNet-50, 학습 2,000장, 시드 0)



Part 1
Normalization

Part 2
Drop out

Part 3
Early
stopping

Part 4
Conclusion

Any Questions?

이름 안 동 균

소속 한양대학교 로봇공학과

이메일 ehdrbs123450@hanyang.ac.kr



*Intelligent Design
for Engineering
Applications Lab*

